Assignment 4.1: BDI Goal-Driven Modeling

ManualMate project report | Student: _____ Course: _____ Date: 02 Nov 2026

What the assignment asks

The agent must store beliefs (what it knows), desires (what it wants to achieve), and intentions (the steps it plans to take). When a belief changes, it must reconsider its intention.

What I implemented

ManualMate keeps these three collections in a BDI agent. During a help request it records the current manual, setup step, recall state, and Agent B safety category. If a leak is reported, it changes the plan to stop the cycle, unplug the appliance, and close the water tap.

Complete BDI source code - manualmate/bdi.py

```
"""Small Belief-Desire-Intention planner used by the ManualMate help flow."""
from __future__ import annotations

from dataclasses import dataclass, field
import logging
from typing import Any

logger = logging.getLogger("manualmate.bdi")

@dataclass
class BDIAgent:
    beliefs: dict[str, Any] = field(default_factory=dict)
    desires: list[dict[str, Any]] = field(default_factory=list)
    intentions: list[str] = field(default_factory=list)
    revision: int = 0

    def observe(self, **updates: Any) -> bool:
        """Update beliefs and re-plan immediately when any belief changes."""
        changed = any(self.beliefs.get(key) != value for key, value in updates.items())
        self.beliefs.update(updates)
        if changed:
            self.revision += 1
            self.replan()
            logger.info("belief update revision=%d replanned desire=%s intentions=%s", self.revision,
                self.desires[0]["name"], self.intentions)
        return changed

    def set_desire(self, name: str, priority: int = 1) -> None:
        self.desires = [{"name": name, "priority": priority}]
        self.replan()

    def replan(self) -> list[str]:
        """Choose an intention list from the current highest-priority beliefs."""
        if self.beliefs.get("recall_matches"):
            self.desires = [{"name": "avoid_recalled_product_risk", "priority": 100}]
            self.intentions = ["pause_repair_guidance", "verify_model_against_recall", "review_official_notice"]
        elif self.beliefs.get("safety_category") == "electrical risk":
            self.desires = [{"name": "avoid_electrical_harm", "priority": 100}]
            self.intentions = ["keep_appliance_unplugged", "do_not_open_electrical_panels",
                "contact_qualified_technician"]
        elif self.beliefs.get("safety_category") == "water leak":
            self.desires = [{"name": "stop_water_leak_safely", "priority": 90}]
            self.intentions = ["stop_cycle", "unplug_appliance", "close_water_tap", "inspect_only_if_safe"]
        elif self.beliefs.get("setup_complete"):
            self.desires = [{"name": "answer_manual_question_safely", "priority": 50}]
            self.intentions = ["find_relevant_manual_passage", "cite_source_page", "check_safety",
                "select_assistance"]
        elif self.beliefs.get("safety_category") == "drainage":
            self.desires = [{"name": "resolve_drainage_safely", "priority": 70}]
            self.intentions = ["keep_towel_and_bucket_ready", "unplug_appliance", "check_drain_path",
                "review_manual_page"]
        else:
            if not self.desires:
                self.desires = [{"name": "complete_setup_safely", "priority": 50}]
            self.intentions = ["review_current_manual_step", "perform_one_step", "confirm_before_next_step"]
        return list(self.intentions)

    def snapshot(self) -> dict[str, Any]:
        return {
            "beliefs": dict(self.beliefs),
            "desires": [dict(item) for item in self.desires],
            "intentions": list(self.intentions),
            "revision": self.revision,
        }
```

Assignment 4.1: Implementation and Test

The BDI agent is called from ManualMate assistance processing. The new safety category is observed as a belief and triggers replanning.

ManualMate integration excerpt - manualmate/assistance.py

```
    agent = BDIAgent({
        "manual_name": manual_name,
        "current_step": step + 1,
        "setup_complete": setup_done,
        "product_query": recall_knowledge.get("product_query", ""),
        "recall_status": recall_knowledge.get("status", "not_checked"),
        "recall_matches": matches,
    })
    agent.set_desire("resolve_manual_request_safely", priority=50)
    agent.observe(safety_category=triage["category"], agent_b_authenticated=ack["payload"].get("authentication")
        == "verified")
```

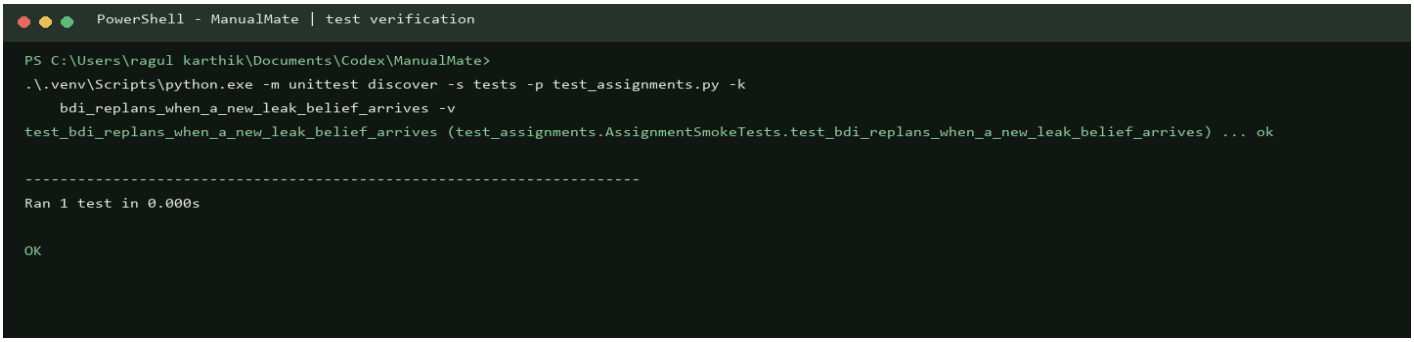
Test used for verification - tests/test_assignments.py

The test begins with the normal setup intention, sends a new water-leak belief, and checks that the intention list changes.

```
    def test_bdi_replans_when_a_new_leak_belief_arrives(self):
        agent = BDIAgent({"setup_complete": False, "safety_category": ""})
        agent.set_desire("complete_setup_safely")
        before = list(agent.intentions)
        agent.observe(safety_category="water leak")
        self.assertIn("review_current_manual_step", before)
        self.assertEqual(agent.intentions[:3], ["stop_cycle", "unplug_appliance", "close_water_tap"])
```

Verification screenshot

Command: .venv\Scripts\python.exe -m unittest discover -s tests -p test_assignments.py -k bdi_replans_when_a_new_leak_belief_arrives -v



Result: Ran 1 test; OK. The test verified that the plan changed from reviewing the current setup step to stopping the cycle, unplugging, and closing the tap.